

LockBox

High-Integrity Local Credential Management System

A Project Report

Submitted in partial fulfillment of the requirements for the course
CSE 3200 — Software Development Project II
6th Semester

by

Md Samiul Islam

ID: 0322320105101001

Simanto Kumar

ID: 0322320105101003

Chanchal Kumar

ID: 0322310105101055

Approved as to style and content by

.....
Mst. Sahela Rahman

Supervisor

Department of Computer Science and Engineering

Pundra University of Science & Technology

Rangpur Road, Gokul, Bogura, Bangladesh.

16 May 2026

DEDICATION

We, the members of this project team, wholeheartedly dedicate this work — the LockBox Local Credential Management System — to our loving parents, our respected supervisor, and all our teachers who have continuously believed in us and supported us through every challenge. Their unwavering encouragement, invaluable guidance, and support for a project focusing on high-level Java security and real-world software development have been instrumental in our journey.

ACKNOWLEDGEMENT

First and foremost, all praises and thanks to Almighty Allah for His countless blessings and grace, which enabled us to complete this project successfully.

We would like to express our sincere and heartfelt gratitude to our respected project supervisor for their valuable guidance, continuous support, and insightful feedback throughout the course of this project. Their direction has been instrumental in shaping our work, and it has been a true privilege to learn and work under their supervision.

We are also profoundly grateful to the Head of the Department of Computer Science and Engineering, Pundra University of Science & Technology, Bogura, for their kind support and for fostering an academic environment conducive to learning and research. Our sincere thanks also extend to all faculty members and staff of the Department of CSE for their assistance and encouragement during this journey.

We would like to acknowledge the support and camaraderie of all our coursemates, whose collaboration and shared discussions have helped in many ways during the completion of this project.

Finally, our deepest gratitude goes to our beloved parents and everyone who directly or indirectly contributed their time, support, and encouragement in helping us accomplish this work.

Md Samiul Islam
Simanto Kumar
Chanchal Kumar

ABSTRACT

LockBox is a standalone desktop application that provides a high-integrity, local-first credential management solution. Designed as a "Zero-Knowledge" utility, it ensures that sensitive data never leaves the user's local hardware, mitigating the risks associated with centralized cloud-based password managers. The system is built using Core Java and Java Swing, utilizing SQLite for localized, serverless storage.

Security is the core focus of LockBox, implemented through a "double-wall" architecture: **PBKDF2** with HMAC-SHA256 (600,000 iterations) for robust key derivation and **AES-256-GCM** for authenticated encryption of at-rest data. The system follows a clean three-layer architecture, separating the Presentation Layer (Java Swing with FlatLaf), Business Logic Layer (Security Core), and Data Access Layer (JDBC-based DAO).

Key functionalities include secure master authentication with account lockout protection, full CRUD operations for credentials, a cryptographically strong password generator, and a secure clipboard manager with automated clearing. LockBox successfully demonstrates the application of advanced cryptographic principles and multi-tier software architecture in a practical, user-centric security tool.

TABLE OF CONTENTS

1. Dedication
2. Acknowledgement
3. Abstract
4. CHAPTER 1: Introduction
 - 1.1 Introduction and Background
 - 1.2 Problem Statement
 - 1.3 Project Objectives and Scope
 - 1.4 Motivation of the Project
5. CHAPTER 2: Literature Review and Feasibility Study
 - 2.1 Introduction
 - 2.2 Review of Existing Systems
 - 2.3 Feasibility Analysis
 - 2.4 System Requirements Specification
6. CHAPTER 3: System Analysis and Requirement Specification
 - 3.1 Introduction
 - 3.2 Functional Requirements
 - 3.3 Non-Functional Requirements
 - 3.4 System Modeling and Flowchart
7. CHAPTER 4: System Design
 - 4.1 Introduction
 - 4.2 System Architecture
 - 4.3 Module Decomposition
 - 4.4 Data Structure and Database Design
 - 4.5 Security Methodology
8. CHAPTER 5: Implementation and Testing
 - 5.1 Introduction
 - 5.2 Implementation Environment and Tools
 - 5.3 Key Implementation Details
 - 5.4 System Testing
9. CHAPTER 6: Conclusion and Future Work
 - 6.1 Conclusion
 - 6.2 Future Work
10. References

CHAPTER 1: INTRODUCTION

1.1 Introduction and Background

In the modern digital landscape, the average individual manages dozens of unique online accounts. This proliferation of credentials has created a significant security challenge, often resulting in "password recycling" or reliance on centralized cloud-based password managers. While cloud solutions offer convenience, they represent a massive single point of failure, as evidenced by major breaches in recent years.

LockBox is designed to return data sovereignty to the user. It is a desktop-based credential management utility that adheres to a "Zero-Knowledge" philosophy. By combining industry-standard Java security APIs with a hardened local SQLite database, LockBox ensures that a user's sensitive information remains exclusively on their local hardware, encrypted with a key that only the user knows.

1.2 Problem Statement

The current password management landscape suffers from three primary issues:

- **Vulnerability of Centralization:** Cloud providers are high-value targets. A single server-side breach can expose millions of user vaults simultaneously.
- **Insecure Local Storage:** Many simple or legacy local managers store data in plaintext or use weak encryption (like Base64) that offers no real protection against local attackers or malware.
- **Browser Dependency:** Browser-integrated managers often store credentials in ways that are easily accessible to other local applications, increasing the risk of credential theft by malicious software.

1.3 Project Objectives and Scope

Primary Objective: To develop a fully functional, local-only digital vault that demonstrates mastery of layered architecture, advanced cryptography (AES-GCM, PBKDF2), and secure GUI design.

Project Scope:

- **Master Authentication:** Registration and login via a Master Password, with account lockout mechanisms.
- **Credential Management:** Full CRUD (Create, Read, Update, Delete) operations for site names, URLs, usernames, and passwords.
- **Password Generation:** A customizable, cryptographically strong generator.

- **Clipboard Safety:** A "Clip-and-Clear" system to prevent password leakage via the system clipboard.
- **Local Storage:** Encrypted SQLite database for data persistence.

1.4 Motivation of the Project

- **Educational:** To deepen understanding of the Java Cryptography Architecture (JCA) and multi-threaded desktop application development.
- **Relevance:** With the rise of cyber-attacks on centralized platforms, there is a growing demand for "offline-first" security tools.
- **Technical Demonstration:** To prove that a highly secure, professional-grade security application can be built using standard Java libraries and disciplined architectural patterns.

CHAPTER 2: LITERATURE REVIEW AND FEASIBILITY STUDY

2.1 Introduction

This chapter examines existing password management solutions and assesses the technical, operational, and economic feasibility of the LockBox project.

2.2 Review of Existing Systems

- **KeePass (Local-Only):** The gold standard for local password management. It is highly secure but often criticized for its complex and dated user interface.
- **Bitwarden (Open Source):** A modern leader that uses PBKDF2 with 600,000+ iterations. LockBox adopts this high-iteration standard for its key derivation process.
- **LastPass (Cloud-Based):** Suffered major breaches in 2022/2023, highlighting the inherent risks of cloud-based storage for sensitive credentials.

Key Findings: Secure password managers must never store the Master Password. Instead, a derived key must be used for encryption, and the vault must be authenticated to prevent tampering.

2.3 Feasibility Analysis

- **Technical Feasibility:** The LockBox feature set can be fully implemented using the Java Standard Library (JDK 25+) and the SQLite JDBC driver. Libraries like FlatLaf provide the modern UI requirements.
- **Operational Feasibility:** As a standalone desktop application, LockBox requires no server setup or network configuration. It is designed to be intuitive for users with basic computer literacy.
- **Economic Feasibility:** The project utilizes open-source tools (OpenJDK, SQLite, Maven, IntelliJ IDEA), resulting in zero software licensing costs.

2.4 System Requirements Specification

Software Requirements:

- Operating System: Windows 10/11, macOS 12+, or Linux.
- Programming Language: Java (JDK 25 recommended).
- Database: SQLite 3.51+.
- Build Tool: Maven 3.9+.

Hardware Requirements:

- RAM: Minimum 2 GB (4 GB recommended).
- Disk Space: 100 MB for the application and database.

CHAPTER 3: SYSTEM ANALYSIS AND REQUIREMENT SPECIFICATION

3.1 Introduction

This chapter formalizes the requirements for LockBox, defining the baseline for validation and testing.

3.2 Functional Requirements

- **FR-01: Master Registration:** Users must set a Master Password on the first run.
- **FR-02: Secure Login:** Authentication via Master Password using PBKDF2 derived keys.

- **FR-03: Account Lockout:** 5-strike lockout with progressive time delays to prevent brute-force attacks.
- **FR-04: Credential CRUD:** Ability to add, view, edit, and delete credentials.
- **FR-05: Masked Display:** Passwords must be masked by default in the dashboard.
- **FR-06: Password Generator:** Generate strong passwords using `SecureRandom`.
- **FR-07: Clipboard Management:** Copy to clipboard with automatic clearing after 30 seconds.
- **FR-08: Vault Search:** Search through site names and usernames in the vault.

3.3 Non-Functional Requirements

- **NFR-01: Security (Zero-Knowledge):** The application must not store the Master Password or any unencrypted credentials.
- **NFR-02: Encryption Standard:** Data at rest must be encrypted using AES-256-GCM.
- **NFR-03: Performance:** Decryption and dashboard loading should occur in less than 1.5 seconds.
- **NFR-04: Usability:** Modern "Dark Mode" interface using the FlatLaf library.
- **NFR-05: Maintainability:** Strict three-layer package separation (`ui`, `security`, `db`).

3.4 System Modeling and Flowchart

LockBox follows a layered logic flow:

1. **Key Derivation:** Master Password + Salt $\rightarrow$ PBKDF2 $\rightarrow$ 256-bit Key.
2. **Vault Access:** Key is used to attempt decryption of the SQLite vault.
3. **Data Operations:** Each entry has its own 12-byte IV for AES-GCM encryption.

CHAPTER 4: SYSTEM DESIGN

4.1 Introduction

This chapter details the architectural blueprint of LockBox, separating the GUI, business logic, and data access.

4.2 System Architecture

LockBox implements a Three-Layer Architecture:

- **Presentation Layer** (`com.lockbox.ui`): Handles the Java Swing interface (LoginFrame, DashboardFrame).
- **Business Logic Layer** (`com.lockbox.security`): The security core managing encryption (`CryptoUtil`) and key derivation (`KeyDerivation`).
- **Data Access Layer** (`com.lockbox.db`): Manages SQLite interactions via JDBC (`VaultDAO`, `DatabaseHelper`).

Layer	Package	Key Classes
Presentation	<code>com.lockbox.ui</code>	<code>LoginFrame</code> , <code>DashboardFrame</code> , <code>PasswordGeneratorDialog</code>
Business Logic	<code>com.lockbox.security</code>	<code>CryptoUtil</code> , <code>KeyDerivation</code>
Data Access	<code>com.lockbox.db</code>	<code>VaultDAO</code> , <code>DatabaseHelper</code> , <code>VaultEntry</code>

4.3 Module Decomposition

- **Security Module**: Implements PBKDF2 with 600,000 iterations and AES-256-GCM.
- **Vault Module**: Handles the mapping of encrypted BLOBs to Java objects (`VaultEntry`).
- **UI Module**: Provides the interactive dashboard and clipboard management.

4.4 Data Structure and Database Design

The SQLite database consists of two primary tables: `vault` and `config`.

Vault Table Schema:

- `id`: INTEGER PRIMARY KEY AUTOINCREMENT
- `site_name`: TEXT (Plaintext for searching)
- `username`: TEXT (Encrypted)
- `password_blob`: BLOB (AES-GCM Ciphertext)
- `iv`: BLOB (12-byte IV)
- `secure_notes`: BLOB (Encrypted notes)
- `secure_notes_iv`: BLOB (IV for secure notes)
- `encrypted_document_content`: BLOB (Encrypted document data)
- `original_file_name`: BLOB (Encrypted filename)
- `original_file_name_iv`: BLOB (IV for filename)
- `password_history_blobs`: BLOB (Historical passwords)
- `password_history_ivs`: BLOB (Historical IVs)

Config Table Schema:

- `key`: TEXT PRIMARY KEY
- `value`: BLOB (Stores Master Password Hash and Salt)

4.6 Browser Integration and Security

To support secure credential access within web browsers, LockBox includes a browser extension that communicates with the desktop application via a Native Messaging host.

- **Architecture:** The extension uses the `nativeMessaging` permission to communicate with a bridge script (`bridge_wrapper.sh`).
- **Security Model:** The browser extension follows a strict "Zero-Knowledge" security posture. It does not store credentials. When the user requests a password for a site, the extension sends a message to the bridge. The LockBox desktop application—which must be running and unlocked—receives the request, performs the decryption using the Master Password derived key, and sends the result back over `stdio`. If the vault is locked, the request is denied, ensuring data remains safe even if the browser is compromised.

CHAPTER 5: IMPLEMENTATION AND TESTING

5.1 Introduction

This chapter details the development environment and the rigorous testing performed to verify the security of the vault.

5.2 Implementation Environment and Tools

- **Language:** Core Java (JDK 25).
- **GUI Framework:** Java Swing + FlatLaf (Dark Theme).
- **Database:** SQLite via Xerial JDBC.
- **Build System:** Maven.
- **IDE:** IntelliJ IDEA.

5.3 Key Implementation Details

- **Atomic Transactions:** `VaultDAO` uses JDBC transactions to ensure data consistency during updates.
- **SecureRandom:** All salts and IVs are generated using `java.security.SecureRandom` for cryptographic strength.
- **Shaded JAR:** The application is packaged into a single executable JAR using the `maven-shade-plugin`.

5.4 System Testing

Unit Testing:

- `CryptoUtilTest`: Verified that tampering with the ciphertext or authentication tag results in an `AEADBadTagException`.
- `KeyDerivationTest`: Verified the speed and consistency of the PBKDF2 process.

Manual Verification:

- **Lockout Test:** Confirmed that 5 failed login attempts trigger the progressive delay.
- **Clipboard Test:** Verified that the clipboard is cleared exactly 30 seconds after a password is copied.
- **Database Inspection:** Confirmed that opening the `.db` file in an SQLite browser shows only unreadable binary data for credentials.

CHAPTER 6: CONCLUSION AND FUTURE WORK

6.1 Conclusion

The LockBox project successfully delivered a high-integrity, local-first password manager. By strictly adhering to "Zero-Knowledge" principles and utilizing modern cryptographic standards (AES-GCM, PBKDF2), the system provides a robust alternative to cloud-based managers. The three-layer architecture ensures maintainability and clear separation of concerns, proving that Core Java is an excellent platform for building sensitive security applications.

6.2 Future Work

- **Browser Extension:** Developing a companion extension to auto-fill credentials safely.
- **Biometric Support:** Integrating Windows Hello or macOS Touch ID for faster unlocking.
- **Encrypted Backups:** Providing a mechanism for secure, manual backups of the vault file.
- **Multi-Factor Authentication (MFA):** Implementing TOTP support within the vault entries.

REFERENCES

1. **NIST.** (2017). *Digital Identity Guidelines: Authentication and Lifecycle Management (Special Publication 800-63B)*. National Institute of Standards and Technology.
2. **OWASP.** (2024). *Password Storage Cheat Sheet*. Open Web Application Security Project.
3. **SQLite.** (2026). *SQLite Documentation*. SQLite Consortium.
4. **Oracle.** (2025). *Java Cryptography Architecture (JCA) Reference Guide*. Oracle Corporation.
5. **FormDev Software.** (2025). *FlatLaf: Flat Look and Feel for Java Swing*. FlatLaf Documentation.
6. **Xerial.** (2026). *sqlite-jdbc: SQLite JDBC Driver for Java*. GitHub Repository.
7. **Bloch, J.** (2018). *Effective Java (3rd ed.)*. Addison-Wesley Professional.

